

Chapter 12 : Handling Errors

Pada bab sebelumnya, Anda telah mempelajari cara memodifikasi data menggunakan Server Actions. Mari kita lihat bagaimana Anda dapat menangani kesalahan dengan baik menggunakan pernyataan JavaScript try/catch dan API Next.js untuk *handling error*.

Bagian 1 : Mengaplikasikan Try/Catch

Ubah kode pada halaman [actions.ts](#), di fungsi createInvoice() dengan kode dibawah ini :

```
TS /app/lib/actions.ts

export async function createInvoice(formData: FormData) {
  const { customerId, amount, status } = CreateInvoice.parse({
    customerId: formData.get('customerId'),
    amount: formData.get('amount'),
    status: formData.get('status'),
  });

  const amountInCents = amount * 100;
  const date = new Date().toISOString().split('T')[0];

  try {
    await sql`
      INSERT INTO invoices (customer_id, amount, status, date)
      VALUES (${customerId}, ${amountInCents}, ${status}, ${date})
    `;
  } catch (error) {
    // We'll also log the error to the console for now
    console.error(error);
    return {
      message: 'Database Error: Failed to Create Invoice.',
    };
  }

  revalidatePath('/dashboard/invoices');
  redirect('/dashboard/invoices');
}
```

Pada fungsi updateInvoice() ikuti kode dibawah ini :

```
ts /app/lib/actions.ts

export async function updateInvoice(id: string, formData: FormData) {
  const { customerId, amount, status } = UpdateInvoice.parse({
    customerId: formData.get('customerId'),
    amount: formData.get('amount'),
    status: formData.get('status'),
  });

  const amountInCents = amount * 100;

  try {
    await sql`
      UPDATE invoices
      SET customer_id = ${customerId}, amount = ${amountInCents}, status = ${status}
      WHERE id = ${id}
    `;
  } catch (error) {
    // we'll also log the error to the console for now
    console.error(error);
    return { message: 'Database Error: Failed to Update Invoice.' };
  }

  revalidatePath('/dashboard/invoices');
  redirect('/dashboard/invoices');
}
```

Dan terakhir pada fungsi deleteInvoice, ikuti kode dibawah ini :

```
ts /app/lib/actions.ts

export async function deleteInvoice(id: string) {
  throw new Error('Failed to Delete Invoice');

  // Unreachable code block
  await sql`DELETE FROM invoices WHERE id = ${id}`;
  revalidatePath('/dashboard/invoices');
}
```

Bagian 2 : Menangani Semua Error

Buatlah file error.tsx didalam folder ini `/dashboard/invoices`,

```
nextjs-dashboard > app > dashboard > invoices > error.tsx
1  'use client';
2
3  import { useEffect } from 'react';
4
5  export default function Error({
6    error,
7    reset,
8  }: {
9    error: Error & { digest?: string };
10   reset: () => void;
11 }) {
12   useEffect(() => {
13     // Optionally log the error to an error reporting service
14     console.error(error);
15   }, [error]);
16
17   return (
18     <main className="flex h-full flex-col items-center justify-center">
19       <h2 className="text-center">Something went wrong!</h2>
20       <button
21         className="mt-4 rounded-md bg-blue-500 px-4 py-2 text-sm text-white transition-colors hover:bg-blue-400"
22         onClick={
23           // Attempt to recover by trying to re-render the invoices route
24           () => reset()
25         }
26       >
27         Try again
28       </button>
29     </main>
30   );
31 }
```

Bagian 3 : Menangani error 404 dengan fungsi notFound

Kalau kalian mengakses halaman yang error pasti akan otomatis masuk ke halaman error tadi, kita akan mengerucut kembali untuk ke error 404. Coba akses link ini,

<http://localhost:3000/dashboard/invoices/2e94d1ed-d220-449f-9f11-f0bbceed9645/edit> =>

Kalian akan masuk ke halaman error tersebut.

```
TS /app/lib/data.ts
export async function fetchInvoiceById(id: string) {
  try {
    // ...

    console.log(invoice); // Invoice is an empty array []
    return invoice[0];
  } catch (error) {
    console.error('Database Error:', error);
    throw new Error('Failed to fetch invoice.');
  }
}
```

Tambahkan kode ini, menyesuaikan dengan root folder yang tertera.

```
TS /dashboard/invoices/[id]/edit/page.tsx

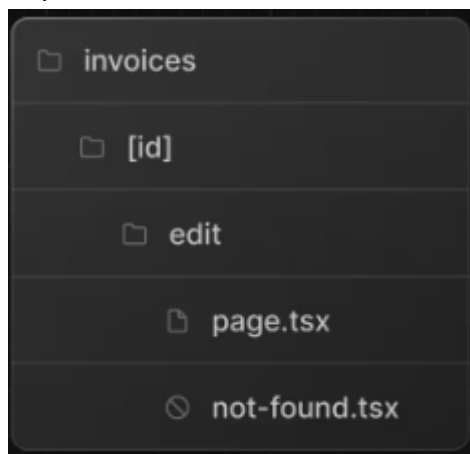
import { fetchInvoiceById, fetchCustomers } from '@app/lib/data';
import { notFound } from 'next/navigation';

export default async function Page(props: { params: Promise<{ id: string }> }) {
  const params = await props.params;
  const id = params.id;
  const [invoice, customers] = await Promise.all([
    fetchInvoiceById(id),
    fetchCustomers(),
  ]);

  if (!invoice) {
    notFound();
  }

  // ...
}
```

Hapus file error.tsx, tambahkan file not-found.tsx dengan root file seperti ini.

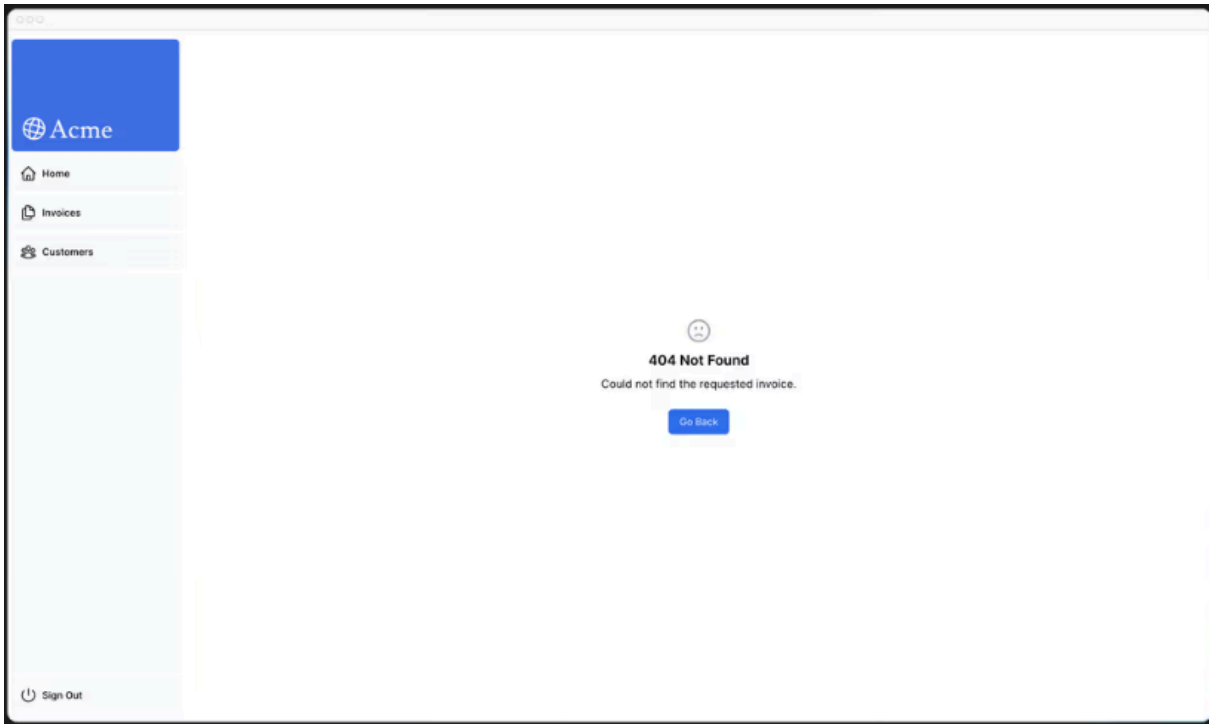


Salin kode ini ke file tersebut.

```
import Link from 'next/link';
import { FaceFrownIcon } from '@heroicons/react/24/outline';

export default function NotFound() {
  return (
    <main className="flex h-full flex-col items-center justify-center gap-2">
      <FaceFrownIcon className="w-10 text-gray-400" />
      <h2 className="text-xl font-semibold">404 Not Found</h2>
      <p>Could not find the requested invoice.</p>
      <Link
        href="/dashboard/invoices"
        className="mt-4 rounded-md bg-blue-500 px-4 py-2 text-sm text-white transition-colors hover:bg-blue-400"
      >
        Go Back
      </Link>
    </main>
  );
}
```

RESULT



CHAPTER 14 : ADDING AUTHENTICATION

Chapter ini, kita akan menambahkan autentikasi pada dashboard.

Tambahkan folder login, dengan root seperti ini /app/login/page.tsx
Salin kode ini.

```
TS /app/login/page.tsx

import AcmeLogo from '@app/ui/acme-logo';
import LoginForm from '@app/ui/login-form';
import { Suspense } from 'react';

export default function LoginPage() {
  return (
    <main className="flex items-center justify-center md:h-screen">
      <div className="relative mx-auto flex w-full max-w-[400px] flex-col space-y-2.5 p-4 md:-mt-32">
        <div className="flex h-20 w-full items-end rounded-lg bg-blue-500 p-3 md:h-36">
          <div className="w-32 text-white md:w-36">
            <AcmeLogo />
          </div>
        </div>
        <div>
          <Suspense>
            <LoginForm />
          </Suspense>
        </div>
      </main>
    );
  }
}
```

Next, kita akan set up [NextAuth.js](#)

Ketikkan ini pada terminal kalian => `pnpm i next-auth@beta`

Kalau mac silahkan ketik ini pada terminal `openssl rand -base64 32`

Kalau windows, masuk ke browser dan akses web ini <https://generate-secret.vercel.app/32>

SALIN KODE YANG KALIAN DAPATKAN UNTUK MENJADI AUTH KEY!

Masuk ke file .env tambahkan

`AUTH_SECRET=SECRET_KEY`

Bagian 2 : Tambahkan Opsi Halaman

Buatkan page `/auth.config.ts` di folder `nextjs-dashboard`

```
1  import type { NextAuthConfig } from 'next-auth';
2
3  export const authConfig = {
4    pages: {
5      signIn: '/login',
6    },
7    callbacks: {
8
9      authorized({ auth, request: { nextUrl } }) {
10        const isLoggedIn = !!auth?.user;
11        const isOnDashboard = nextUrl.pathname.startsWith('/dashboard');
12        if (isOnDashboard) {
13          if (isLoggedIn) return true;
14          return false; // Redirect unauthenticated users to login page
15        } else if (isLoggedIn) {
16          return Response.redirect(new URL('/dashboard', nextUrl));
17        }
18        return true;
19      },
20    },
21    providers: [], // Add providers with an empty array for now
22  } satisfies NextAuthConfig;
```

Bagian 3 : Password Hashing

Buatkan page `/auth.ts` di folder `nextjs-dashboard`

```
1  import NextAuth from 'next-auth';
2  import { authConfig } from './auth.config';
3  import Credentials from 'next-auth/providers/credentials';
4  import { z } from 'zod';
5  import type { User } from '@app/lib/definitions';
6  import bcrypt from 'bcryptjs';
7  import postgres from 'postgres';
8
9  const sql = postgres(process.env.POSTGRES_URL!, { ssl: 'require' });
10
11  Windsurf: Refactor | Explain | Generate JSDoc | X
12  async function getUser(email: string): Promise<User | undefined> {
13    try {
14      const user = await sql<User[]>`SELECT * FROM users WHERE email=${email}`;
15      return user[0];
16    } catch (error) {
17      console.error('Failed to fetch user:', error);
18      throw new Error('Failed to fetch user.');
19    }
20  }
```

```

export const { auth, signIn, signOut } = NextAuth({
  ...authConfig,
  providers: [
    Credentials({
      Windsurf: Refactor | Explain | Generate JSDoc | X
      async authorize(credentials) {
        const parsedCredentials = z
          .object({ email: z.string().email(), password: z.string().min(6) })
          .safeParse(credentials);

        if (parsedCredentials.success) {
          const { email, password } = parsedCredentials.data;
          const user = await getUser(email);
          if (!user) return null;
          const passwordsMatch = await bcrypt.compare(password, user.password);

          if (passwordsMatch) return user;
        }

        console.log('Invalid credentials');
        return null;
      },
    }),
  ],
});

```

Bagian 4 : Ubah Form Login

Ubah terlebih dahulu di file [actions.ts](#)

```

/app/lib/actions.ts

'use server';

import { signIn } from '@/auth';
import { AuthError } from 'next-auth';

// ...

export async function authenticate(
  prevState: string | undefined,
  formData: FormData,
) {
  try {
    await signIn('credentials', formData);
  } catch (error) {
    if (error instanceof AuthError) {
      switch (error.type) {
        case 'CredentialsSignin':
          return 'Invalid credentials.';
        default:
          return 'Something went wrong.';
      }
    }
    throw error;
  }
}

```

Tambahkan ini di halaman login-form.tsx (yang ditandai biru yah)

TS app/ui/login-form.tsx

```

use client';

import { lusitana } from '@app/ui/fonts';
import {
  AtSymbolIcon,
  KeyIcon,
  ExclamationCircleIcon,
} from '@heroicons/react/24/outline';
import { ArrowRightIcon } from '@heroicons/react/20/solid';
import { Button } from '@app/ui/button';
import { useActionState } from 'react';
import { authenticate } from '@app/lib/actions';
import { useSearchParams } from 'next/navigation';

export default function LoginForm() {
  const searchParams = useSearchParams();
  const callbackUrl = searchParams.get('callbackUrl') || '/dashboard';
  const [errorMessage, formAction, isPending] = useActionState(
    authenticate,
    undefined,
  );

  return (
    <form action={formAction} className="space-y-3">
      <div>
        <keyIcon className="pointer-events-none absolute left-3 top-1/2 h-[18px] w-[18px] -translate
      </div>
    </div>
    </div>
    <input type="hidden" name="redirectTo" value={callbackUrl} />
    <button className="mt-4 w-full" aria-disabled={isPending}>
      Log in <ArrowRightIcon className="ml-auto h-5 w-5 text-gray-50" />
    </button>
    <div
      className="flex h-8 items-end space-x-1"
      aria-live="polite"
      aria-atomic="true"
    >
      {errorMessage && (
        <div>
          <ExclamationCircleIcon className="h-5 w-5 text-red-500" />
          <p className="text-sm text-red-500">{errorMessage}</p>
        </div>
      )}
    </div>
    </div>
    </form>
  );
}

```

Bagian 5 : Tambahkan fungsi logout

Ubah sidenav.tsx sesuai kode ini.

```
TS /ui/dashboard/sidenav.tsx

import Link from 'next/link';
import NavLinks from '@app/ui/dashboard/nav-links';
import AcmeLogo from '@app/ui/acme-logo';
import { PowerIcon } from '@heroicons/react/24/outline';
import { signOut } from '@auth';

export default function Sidenav() {
  return (
    <div className="flex h-full flex-col px-3 py-4 md:px-2">
      // ...
      <div className="flex grow flex-row justify-between space-x-2 md:flex-col md:space-x-0 md:space-y-2">
        <NavLinks />
        <div className="hidden h-auto w-full grow rounded-md bg-gray-50 md:block"></div>
        <form
          action={async () => {
            'use server';
            await signOut({ redirectTo: '/' });
          }}
        >
          <button className="flex h-[48px] grow items-center justify-center gap-2 rounded-md bg-gray-50 p-
            <PowerIcon className="w-6" />
            <div className="hidden md:block">Sign out</div>
          </button>
        </form>
      </div>
    </div>
  );
}
```

CHAPTER 15 : ADDING METADATA

Bagian 1 : Judul Halaman & Deskripsi

Tambahkan kode ini pada halaman layout.tsx

```
import { Metadata } from 'next';

export const metadata: Metadata = {
  title: {
    template: '%s | Acme Dashboard',
    default: 'Acme Dashboard',
  },
  description: 'The official Next.js Learn Dashboard built with App Router.',
  metadataBase: new URL('https://next-learn-dashboard.vercel.sh'),
};
```

Terakhir, tambahkan lagi di halaman invoices.

```
import { Metadata } from 'next';

export const metadata: Metadata = {
  title: 'Invoices',
};
```